

学习构建环境：通过可验证环境合成实现自演化推理 RL

Yucheng Shi¹, Zhenwen Liang¹, Kishan Panaganti¹, Dian Yu¹, Wenhao Yu¹, Haitao Mi¹

¹Tencent HY LLM

我们追求一种自我改进语言模型的愿景：模型不只是生成要模仿的问题或轨迹，而是构造训练自身的环境。在零数据推理 RL 中，这把自我改进从数据生成循环重新表述为环境构造循环，其中每个工件都是可复用的可执行对象，能够采样实例、计算参考答案并为响应打分。这一愿景能否持续带来改进，取决于一个性质：环境必须呈现稳定的求解-验证不对称性，即模型必须能够一次性写出一个 oracle，而它又不能在新的实例上可靠地用自然语言执行同一过程。这种不对称性有两种互补形式。有些任务在推理上算法性地困难，但作为代码却很平凡：一个动态规划或图遍历程序一旦编译，就能产生无界数量的校准实例。另一些任务本身难以求解但易于验证，例如种植的子集和或约束满足。二者都会在提出与求解之间形成持久间隙，策略无法通过操纵验证器来关闭这个间隙；正是这个间隙在学习者进步时持续保持奖励的信息量。我们在 EvoEnv 中实例化这一观点：这是一种单策略生成器-求解器方法，从十个种子合成 Python 环境，并且只有在通过分阶段验证、语义自审查、相对于求解器的难度校准和新颖性检查之后才接纳它们。最有力的证据来自已经很强的模型状态：在 Qwen3-4B-Thinking 上，固定公共数据 RLVR 和固定手工环境 RLVR 都降低了平均分，而 EvoEnv 将其从 72.4 提升到 74.8，相对增益为 3.3%。我们认为，稳定的自我改进不依赖于生产更多合成数据，而依赖于模型学会构造那些难度在结构上始终超出自身可及范围的世界。

Date: 2026 年 7 月 1 日

1 引言

语言模型自我改进常被表述为一个数据生成问题：模型在其当前能力边界附近产生更多问题、轨迹、解答或困难样例 [13, 18, 19]。这一表述有用但并不完整。有能力的 LLM 并不只是通过模仿更多样例而改进；它们还通过与环境交互而改进，这些环境会生成情境、施加约束，并通过执行、工具、测试或状态变化返回反馈 [45, 32]。我们提出的问题是，语言模型能否不仅从自生成样例中学习，还能从自构建的训练环境中学习。我们在一个刻意受控的设置中研究这一问题：面向推理的零数据可验证奖励强化学习 (RLVR)。一个推理环境是带有四个例程的可复用可执行工件：采样器生成潜在任务实例，预言机计算参考答案，渲染器将实例转换为自然语言提示，评分器评估求解器响应。模型可以编写这一工件，但一旦该工件通过验证并被接纳进入池中，它的奖励就由执行决定，而不是由模型当前采样出的答案决定。

这一区分回应了现有自训练配方的两个局限。标准 RLVR 从固定数据集、答案等价规则、单元测试或可执行检查器中获得可靠监督 [10, 42, 21]。然而，已验证分布是固定的。随着策略改进，许多提示几乎总是会被解出，另一些提示几乎总是失败，而组相对方法会失去有用的奖励变化。在一个饱和池上继续优化，随后可能收窄行为或导致遗忘，而不是扩展能力 [11, 38, 29]。因此，稳定的验证器并不充分；已验证分布还必须保持在求解器不断变化的能力边界附近。

逐实例自博弈提供了适应性，但可能损害标签稳定性。如果新提示由自一致性、多数投票、语义聚类或另一个来自同一正在优化的策略的信号来标注，奖励函数就会随学习者一起移动 [13, 18, 19]。如果模型转而而为单个问题生成一次性可执行验证器，正确性会有更好的根据，但该工件在一次采样

展开之后就被消耗掉 [51]. 由此得到的系统仍然缺少一种持久对象: 它可以被一次验证、反复采样、针对当前求解器进行校准、在饱和时退役, 并在之后作为种子材料复用.

我们的论点是, 自合成的单位应当是环境, 而不是单问题. 生成的问题给出一个提示和一个标签; 生成的环境给出一个提示分布和一个可复用的可执行奖励来源. 使这种环境层级单位可处理的原因, 不仅是跨采样展开的摊销, 还在于一种我们称为稳定的求解-验证不对称性的结构性性质: 在广泛的推理任务中, 编写或检查一个可执行过程, 比在新实例上执行相应的自然语言推理过程更容易.

这种不对称性以两种互补形式出现. 在算法型任务中, 例如动态规划、图遍历、模递推、排序和序列计算, 模型也许能够写出紧凑的预言机, 即便它不能在任意渲染实例上用自然语言可靠地执行该算法. 在验证型任务中, 例如植入式子集和、可行性检查和约束满足, 产生一个有效答案可能很难, 而检查一个候选答案则很简单. 两种形式都在提出与求解之间产生了持久间隙, 策略无法通过操纵验证器来关闭这一间隙, 因为验证器是冻结代码. 正是这一间隙使奖励在学习者改进时仍然具有信息量, 也正是它区分了自构建环境与自生成问题.

我们在 EvoEnv 中实例化这一观点, 这是一个单策略双角色训练器. 同一个策略在生成器角色和求解器角色之间交替: 前者从一个小种子池提出 Python 环境, 后者回答从已接纳环境池中采样的新提示. 候选环境只有在满足四项契约时才进入池中: 它们在严格接口下执行; 其预言机和评分器在保守的语义自我审查下与所声明的任务匹配; 采样实例对当前策略而言有难度但可解; 并且池保持足够宽, 以避免模板坍缩. 我们通过 L1-L5 验证、语义自我审查、新颖性门控、批内去重和池轮换来强制执行这些契约.

这一表述最强的证据来自已经很强的模型体制. 在 Qwen3-4B-Thinking 上, 固定公共数据 RLVR 和固定手工构造环境 RLVR 都降低了表 1 中的平均分数, 而 EvoEnv 提高了它. 这一结果表明, 自演化环境不只是为弱模型获得更多合成数据的一种方式. 它们是在静态分布已经饱和或已经与学习者错位时, 让奖励保持稳定且与能力边界校准的一种方式. 我们的贡献如下:

1. 我们为零数据推理 RL 形式化可验证环境合成, 并将算法型和验证型两种形式中的稳定求解-验证不对称性识别为一种结构性性质, 它使自构建环境能够作为持久奖励来源, 而不是与策略耦合的伪标签.
2. 我们提出 EvoEnv, 一种单策略提出者-求解器算法, 它通过分阶段验证、语义自我审查、相对于求解器的难度校准、新颖性控制和池轮换来策划自生成环境池.
3. 我们表明 EvoEnv 改进了三个模型家族, 尤其是改进了一个已经很强的思维模式检查点, 而固定公共数据 RLVR 和固定手工构造环境 RLVR 在该检查点上都会降低平均性能.

2 相关工作

我们通过两个问题来定位 EvoEnv: 训练期间被复用的对象是什么, 以及它的奖励来自哪里? 既有工作提供了稳定验证器、自适应自生成课程、可执行任务检查或环境层级训练. EvoEnv 在一个特定设置中组合这些要素: 零数据推理 RL, 其中学习者自己编写可复用可执行环境, 且每个环境只有经过验证、相对于求解器的校准和新颖性过滤之后才会被接纳.

固定分布 RLVR. 有验证器支撑的 RL 对推理高度有效, 因为结果奖励可以不借助学习到的偏好模型来计算 [10, 42, 21]. 然而, 大多数 RLVR 系统在固定提示和检查器集合上训练. 这很可靠, 但并不自适应: 一旦样例几乎总是被解出或几乎总是失败, 组相对奖励就会失去有用变化, 持续优化可能导致饱和、过度专门化或遗忘 [11, 38, 29]. EvoEnv 保留了 RLVR 的关键收益-执行扎根的奖励-但用一个不断变化的已验证可执行环境池替换静态提示池.

自生成课程. 一条宽泛的自我改进方法路线在模型生成的推理依据、问题、偏好或判断上训练 [44,

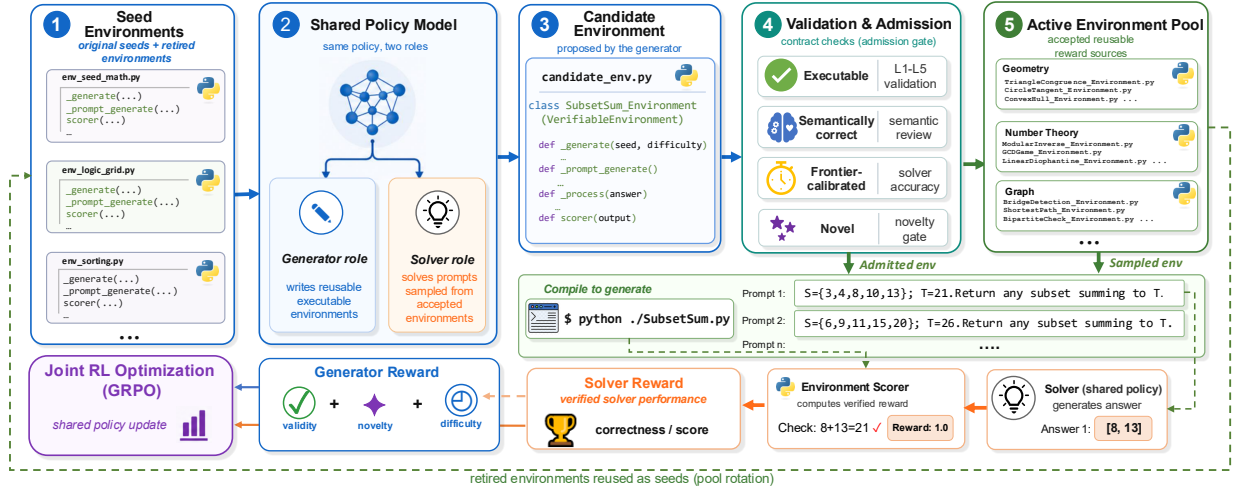


图 1 EvoEnv 概览. 单一策略生成可复用可执行环境, 并求解从已接纳池中采样的新实例. 候选环境只有经过验证、语义审查、相对于求解器的难度校准和新颖性过滤之后才进入池中.

30, 7, 43, 12]. 近期零数据方法更进一步, 让模型提出自己的任务, 并通过多数投票、自一致性、内部反馈或共同演化的智能体来估计正确性 [13, 9, 4, 56, 48, 24, 52, 16, 49, 35, 5]. 这些方法具有适应性, 但它们的奖励信号常常与策略耦合: 学习的同一个模型家族也帮助决定什么是正确的. EVOL-RL 等基于新颖性的变体缓解了这些循环中的坍缩 [54]. EvoEnv 共享自适应自我改进的目标, 但它不把模型采样出的答案当作标签来信任; 它只信任通过准入并随后被冻结用于评分的可执行工件.

可执行扎根. 若干方法通过代码或测试来扎根自生成任务. Absolute Zero Reasoner 通过执行验证程序/输入/输出任务 [51]; Self-Challenging Agents 生成 Code-as-Task 验证器 [53]; SPC 为推理错误训练对抗性批评器 [3]; 智能体系统则把类似思想扩展到工具使用和软件工程 [40, 55, 37]. 这些工作更接近我们, 因为正确性不再只是自我相信. 主要差异在于复用单位: 它们通常生成任务、测试或回合, 而 EvoEnv 生成环境层级对象, 该对象可以采样许多新实例, 针对当前求解器校准, 在饱和时退役, 并作为种子材料复用.

环境层级训练. 最接近的邻近工作也把环境视为训练对象. 一些环境是外部给定的, 例如固定规则游戏、语言游戏、文档语料库或手工编写的可验证 Python 套件 [18, 20, 15, 22, 19, 46]. 另一些工作通过离线流水线合成工具、网页、UI 或具身环境 [31, 39, 50, 2, 25, 17]. 学习型模拟器方法则使用 LLM 世界模型, 因而奖励依赖模拟器的信念 [6, 36, 8]. EvoEnv 研究一个更受控的情形: 同一个策略既求解推理任务, 也编写确定性 Python 环境, 而准入是在线策略的、经过求解器校准的, 并且以执行为根据, 而不是由人手工编写、离线流水线生成或由模拟器定义. 附录 A 给出了更完整的比较.

3 方法

EvoEnv 训练一个单一策略来做两件相关的事: 求解可验证推理任务, 以及构建这些任务被采样自其中的可执行环境. 因此, 中心对象不是生成的问题, 而是可复用的环境. 候选环境只有在通过机械验证、保守语义审查、相对于求解器的难度校准和新颖性过滤之后, 才允许影响求解器训练. 设计目标很简单: 教会模型构建新的训练世界, 同时确保用于求解器更新的奖励来自冻结执行, 而不是来自模型当前采样出的答案. 完整工作流程见图 1 和算法 1.

3.1 环境接口

可验证环境是一个可复用可执行对象，而不是单个带标签的问题。我们将其写作 $e = (G_e, \Pi_e, S_e)$ 。给定种子 σ 和难度参数 δ ，生成器-预言机例程产生一个潜在实例和参考答案， $(z, a) = G_e(\sigma, \delta)$ 。求解器只观察渲染后的提示 $x = \Pi_e(z)$ ，并因其响应 y 获得奖励 $R_e(y; \sigma, \delta) = S_e(y, z, a)$ 。因此，一旦环境被接纳，奖励来源就是一条冻结的可执行路径：对于固定的 e, σ, δ ，当前策略只能改变采样出的响应，不能改变参考答案或评分规则。

图 2 用一个最小排序环境实例化了这一接口。该环境采样一个数组，用 `sorted` 计算可执行参考答案，将数组渲染为面向模型的提示，并通过精确比较对解析后的响应评分。这展示了贯穿 `EvoEnv` 的操作求解-验证不对称性：编写并运行这段代码相对容易，但求解许多新渲染实例仍然为语言模型提供非平凡训练信号。

在实现中，每个候选都作为 `Verifiable Environment` 的 Python 子类被输出。方法 `_generate(seed, difficulty)` 通过构造 z 并计算 a 来实现 G_e ；`_prompt_generate` 实现 Π_e ；而 `_process` 连同 `scorer` 实现 S_e 。候选被限制在获准的标准库子集内，并在带有墙钟超时的沙箱子进程中执行。

这一接口同时覆盖等值检查型预言机和可行性检查型预言机，前者例如排序、动态规划、图遍历、模递推和序列计算，后者例如植入式子集和与约束满足。可行性任务需要更强的评分器探测，因为可能有多个有效答案。附录 B 给出完整数据流图，附录 D 提供一个完整的植入式子集和示例。

3.2 验证与语义审查

我们的方法配备多层验证。令 $\ell(e)$ 表示候选 e 达到的最高验证层。L1 提取可解析的 Python，并检查预期类和方法是否存在。L2 实例化该类，并在若干种子和难度设置上运行生成、提示渲染、解析和评分。L3 通过在相同种子下重复生成并比较潜在实例、提示和参考对象来检查确定性。L4 通过要求跨种子和难度值存在变化来检查非平凡性。L5 检查局部评分器契约：存储的参考对象必须得到正分；注入的扰动、畸形答案和类型不匹配答案不能得到正分；并且解析不得泄漏隐藏参考对象。只有 L5 候选才进入语义审查和相对于求解器的校准。

单靠机械执行无法证明生成代码实现了提示中描述的任务。候选可以是确定性的且非平凡的，同时却计算错误的递推、奖励错误的目标或接受畸形答案。因此，我们加入一个保守的语义审查过滤器。审查器接收候选源代码、一个或多个具体生成实例、参考对象、渲染后的提示和评分器探测。它被要求执行一项代码审查任务：跟踪所声明的任务，检查生成器是否计算了所声明的量，并搜索领域重标记、隐藏答案泄漏和过度宽松的解析。

审查器是训练中其他位置使用的同一策略，但它的裁决不被用作生成器奖励。这一区分很重要。生成器的标量奖励由机械验证、相对于求解器的难度和新颖性计算得到；语义审查器只决定一个在其他方面有效的候选是否可以进入主动求解器训练池。被拒绝的候选仍可贡献其机械计算得到的生成器采样展开奖励，但它不能成为求解器训练的奖励来源。这移除了生成器可被优化来满足审查器自然语言偏好的直接通道。

```

1 import random
2
3 class SortingEnv(VerifiableEnvironment):
4
5     def _generate(self, seed, difficulty):
6         #  $G_e$ 
7         rng = random.Random(seed)
8         z = [rng.randint(0, 99)
9              for _ in range(difficulty)]
10        return z, sorted(z)
11        #  $(z, a)$ 
12
13    def _prompt_generate(self, z):
14        #  $\Pi_e$ 
15        arr = " ".join(map(str, z))
16        return "Sort: " + arr
17        #  $x$ 
18
19    def _process(self, y):
20        try:
21            return list(map(int, y.split()))
22        except ValueError:
23            return None
24
25    def scorer(self, y, z, a):
26        #  $S_e$ 
27        return float(self._process(y) == a)

```

图 2 一个最小排序环境。方法 `_generate` 实现 G_e ，`_prompt_generate` 实现 Π_e ，而 `_process` 与 `scorer` 共同实现 S_e 。

我们运行 $K_{\text{rev}} = 3$ 次独立审查, 并使用任一拒绝规则: 如果任何审查识别出可能的语义错误, 候选就被拒绝进入池. 审查任务比从零开始求解基准问题局部得多: 审查器看到源代码、隐藏状态、参考对象和评分器行为, 只需检查它们之间的一致性. 作为额外的健全性检查, 我们用更强的外部审查器审计这种同策略审查, 并发现高度一致; 细节见附录 E.

3.3 难度与新颖性奖励

对每个 L5 候选, 我们估计其采样实例是否为当前求解器产生有用的结果变化. 我们通过运行 G_e 采样 $m = 8$ 个校准实例, 对每个实例抽取一个求解器响应, 并取平均:

$$\hat{a}_m(e; \pi_\theta) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}[S_e(y_i, z_i, a_i) > 0], \quad (z_i, a_i) = G_e(\sigma_i, \delta), \quad y_i \sim \pi_\theta(\cdot \mid \Pi_e(z_i)). \quad (1)$$

我们对每个实例使用一个响应来估计通过率, 而不膨胀计算量. 对当前求解器而言, $\hat{a}_m(e; \pi_\theta) = 0$ 的候选过难、说明不足或过于严格; $\hat{a}_m(e; \pi_\theta) = 1$ 的候选则已饱和或过于宽松. 因此, 我们要求 $0 < \hat{a}_m(e; \pi_\theta) < 1$ 才能接纳. 生成器的难度奖励使用一个分段调度, 按其相对于求解器的通过率给每个 L5 候选打分:

$$Q_{\text{unc}}(e; \theta) = \exp\left(-\frac{(\hat{a}_m(e; \pi_\theta) - a^*)^2}{2\sigma_a^2}\right), \quad a^* = 0.3. \quad (2)$$

目标 $a^* = 0.3$ 将生成偏向于可解但仍有挑战性的环境: 当前求解器成功得足够频繁, 可以产生正样例, 但失败也足够频繁, 可以保留有用的奖励变化. 我们选择低于 0.5 的目标, 因为接近一半准确率的候选可能随着求解器改进而快速饱和.

新颖性防止生成器反复产生第一个通过验证的模板. 我们用冻结的外部嵌入模型 all-MiniLM-L6-v2 [26] 嵌入每个环境, 而不是使用训练策略本身. 每个候选有两个嵌入: 来自其 `prompt_template` 的提示嵌入 u_e , 以及来自其清洗后 `_generate` 主体的代码嵌入 v_e . 令 $\mathcal{C}_p$ 和 $\mathcal{C}_c$ 为先前已接纳环境的提示和代码嵌入缓存. 我们计算

$$\text{sim}_t(e) = \lambda \max_{u' \in \mathcal{C}_p} \cos(u_e, u') + (1 - \lambda) \max_{v' \in \mathcal{C}_c} \cos(v_e, v'), \quad \lambda = 0.5, \quad (3)$$

并定义 $N_t(e) = 1 - \text{sim}_t(e)$. 双视角新颖性分数是一道防护栏, 用来防止表层重复. 仅用提示嵌入可能漏掉带有不同故事包装的代码克隆, 仅用代码嵌入也可能漏掉用不同语言写出的同一任务. 结合提示和代码视角会降低精确或近似精确重复的吸引力. 同时, 我们也承认, 当表层变体在语义上有效并向求解器呈现不同自然语言语境时, 它们仍然可能是有用的训练环境. 新颖性权重会随已接纳流的重复程度而调整. 令 $\bar{s}_t$ 为批内最大相似度的指数移动平均. 我们设置

$$\gamma_t = \gamma_{\min} + (\gamma_{\max} - \gamma_{\min}) \cdot \text{clip}\left(\frac{\bar{s}_t - \eta_{\text{low}}}{\eta_{\text{high}} - \eta_{\text{low}}}, 0, 1\right). \quad (4)$$

因此, 当已接纳池变得重复时, 探索压力上升; 当新环境已经多样时, 探索压力放松. 完整生成器奖励把分层验证与新颖性奖励结合起来:

$$R_{\text{gen}}(e; \theta, t) = Q_{\text{val}}(\ell(e), \hat{a}_m(e; \pi_\theta)) + \mathbb{I}[\ell(e) \geq 2] \gamma_t N_t(e). \quad (5)$$

其中验证项为

$$Q_{\text{val}}(\ell, \hat{a}) = -\mathbb{I}[\ell < 1] - \frac{1}{2}\mathbb{I}[\ell = 1] - \frac{1}{4}\mathbb{I}[\ell = 2] + \mathbb{I}[\ell = 5] Q_{\text{unc}}(\hat{a}). \quad (6)$$

验证项惩罚机械上无效的候选, 对通过执行层级检查但未通过最高层验证的候选赋予零奖励, 并且只把相对于求解器的不确定性奖励 Q_{unc} 给 L5 候选. 新颖性奖励由 $\ell(e) \geq 2$ 门控, 因此不可解析或语法破损的候选不能仅靠新颖性恢复. 关键的是, 语义审查裁决不是 R_{gen} 中的一项; 它只影响入池决定.

3.4 池接纳与联合优化

候选只有在满足全部四项接纳要求时才会被插入主动求解器训练池:

$$A_t(e) = \mathbb{I}[\ell(e) = 5] \cdot \mathbb{I}[\text{review}(e) = 1] \cdot \mathbb{I}[0 < \hat{a}_m(e; \pi_\theta) < 1] \cdot \mathbb{I}[\text{sim}_t(e) < \tau_{\text{gate}}]. \quad (7)$$

已接纳环境进入 $\mathcal{P}_t$, 并且之后既可用于求解器训练, 也可作为 $\mathcal{S}_t$ 中的示例用于未来生成器提示. 在每个训练步骤中, 我们形成求解器和生成器采样展开组. 一个求解器组采样一个环境 $e \sim \mathcal{P}_t$, 用 G_e 和 Π_e 生成实例与提示, 从 π_θ 中采样多个回答, 并用冻结的环境评分器 S_e 为每个回答打分. 因此, 求解器接收普通的、由验证器支撑的 RL 反馈: 只有当它生成被环境接受的答案时才会获得奖励.

一个生成器组从同一个由 $\mathcal{S}_t$ 构造的 few-shot 生成器提示中采样多个候选环境. 每个候选环境获得式 equation 5 中的奖励; 满足式 equation 7 的候选还会被加入活动池. 我们分别在求解器组和生成器组内部归一化优势, 然后用一个按角色条件化的 GRPO 目标优化共享策略:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{solver}}^{\text{GRPO}}(\theta) + w_{\text{gen}} \mathcal{L}_{\text{gen}}^{\text{GRPO}}(\theta) + \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}), \quad w_{\text{gen}} = 0.3. \quad (8)$$

两个角色使用不同提示, 但共享参数. 我们出于效率考虑选择单策略设计: 同一个从可执行推理环境中受益的模型, 也学习构造这些环境所需的代码模式. 在实践中, 独立的组内归一化和生成器权重 w_{gen} 会防止较长且高方差的生成器轨迹主导求解器更新.

池轮换控制长时程课程. 当一个环境已经被用于固定数量的求解器 epoch 后, 它会从 $\mathcal{P}_t$ 中退役, 并且不再被采样用于直接求解器训练. 退役环境可以转而进入 $\mathcal{S}_t$, 作为未来生成器提示的 few-shot 示例. 原始种子环境受到下限 $\rho_{\min} = 0.2$ 的保护: 如果某个原始种子的退役会使活动池中原始环境占比低于该下限, 那么它就不会退役. 这在保持生成器上下文锚定的同时, 也允许求解器训练池向新合成的环境迁移.

4 实验

我们的实验检验三个问题. (Q1) 与未训练基座和强 RLVR 基线相比, EvoEnv 是否能提升下游推理能力? (Q2) 环境合成能否在训练过程中持续提供前沿信号, 还是会像固定池一样饱和? (Q3) 哪些组件是承重组件?

4.1 设置

模型 我们在三个多样化的基座模型上评估: **Qwen3-4B-Instruct-2507** [41], 一个标准的 4B 指令微调模型; **Qwen3-4B-Thinking-2507** [41], 一个已经较强的思考模式检查点; 以及 **Nemotron-Cascade-8B-IFRL** [33], 一个来自不同家族的 8B 思考模型.

种子与训练 所有运行都从同一组固定的十个种子环境开始, 这些环境覆盖排序、动态规划、图和数

算法 1 EvoEnv 的一次自进化迭代。

输入: 策略 π_{θ_t} ; 活动池 $\mathcal{P}_t$; 种子池 $\mathcal{S}_t$; 新颖性缓存 $\mathcal{C}_t$
输出: 更新后的策略 $\pi_{\theta_{t+1}}$ 与池 $(\mathcal{P}_{t+1}, \mathcal{S}_{t+1})$

- 1: 初始化生成器数据 $\mathcal{D}_t^g \leftarrow \emptyset$ 、求解器数据 $\mathcal{D}_t^s \leftarrow \emptyset$ 、已接受集合 $\mathcal{A}_t \leftarrow \emptyset$
- 2: 采样候选环境 $\{e_i, \tau_i^g\}_{i=1}^M \sim \pi_{\theta_t}(\cdot \mid \text{Prompt}_{\text{gen}}(\mathcal{S}_t))$
- 3: **for** $i = 1, \dots, M$ **do**
- 4: $(r_i^g, A_i) \leftarrow \text{EvalEnv}(e_i; \pi_{\theta_t}, \mathcal{C}_t)$
- 5: $\mathcal{D}_t^g \leftarrow \mathcal{D}_t^g \cup \{(\tau_i^g, r_i^g)\}$; 若 $A_i = 1$, 则将 e_i 加入 $\mathcal{A}_t$
- 6: **end for**
- 7: **for** $j = 1, \dots, B$ **do**
- 8: 采样 $e \sim \mathcal{P}_t \cup \mathcal{A}_t$ 、 $(z, a) \sim G_e$ 、 $x = \Pi_e(z)$, 以及回答 $\{y_{jm}\}_{m=1}^K \sim \pi_{\theta_t}(\cdot \mid x)$
- 9: $\mathcal{D}_t^s \leftarrow \mathcal{D}_t^s \cup \{(\tau_{jm}^s, S_e(y_{jm}, z, a))\}_{m=1}^K$
- 10: **end for**
- 11: $\theta_{t+1} \leftarrow \text{GRPO}(\theta_t; \mathcal{D}_t^g, \mathcal{D}_t^s)$
- 12: $(\mathcal{P}_{t+1}, \mathcal{S}_{t+1}, \mathcal{C}_{t+1}) \leftarrow \text{UpdatePool}(\mathcal{P}_t, \mathcal{S}_t, \mathcal{C}_t, \mathcal{A}_t)$

论模板 (附录 F)。我们不使用任何外部问题-答案数据。我们使用单策略 GRPO、生成器优势缩放 $w_{\text{gen}} = 0.3$, 以及附录 I 中的超参数进行训练。所有模型都训练 100 步。

评估 我们使用 Nemo-Skills 工具包 [23], 在八个基准上报告 pass@1 [%]。对于小规模题集的竞赛数学基准, 即 AIME 2024、AIME 2025、HMMT Feb 2025、Beyond-AIME [28] 和 Brumo 2025 [1], 我们对 16 次运行取平均以降低方差; AMC 2023 [1]、GPQA Diamond [27] 和 LiveCodeBench v6, 2024-08–2025-05 (LCB) [14] 则报告单次运行的 pass@1。除非另有说明, 评估使用最长 128k 上下文、符号等价检查、温度 0.6 和 top-p 0.95。

基线 我们比较以下基线: **未训练**, 即在相同解码协议下的基座检查点; **R-Zero** [13], 一种零数据的 proposer-solver RLVR 方法; **DAPO** [42], 在 DAPO-Math-17k 上以匹配计算量训练的 GRPO; 以及 **RLVE** [46], 一种使用人工策划环境并匹配训练步数的固定环境 RLVR。

4.2 主要结果：固定分布失效时，自适应环境仍然有用

表 1 给出了主要结果。最右列报告每一行在各基准上的平均 pass@1。这里有三个突出模式。第一, EvoEnv 是唯一能提升全部三个模型家族的方法。它把 Qwen3-4B-Instruct 的平均值从 49.2 提高到 53.1, 把已经较强的 Qwen3-4B-Thinking 检查点从 72.4 提高到 74.8, 并把来自不同家族的思考模型 Nemotron-Cascade-8B 从 71.0 提高到 73.2。这些分别对应 7.9%、3.3% 和 3.1% 的相对增益。第二, 强模型场景凸显了固定课程的弱点。在 Qwen3-4B-Thinking 上, DAPO 和 RLVE 将平均值降至 64.8 和 69.2, 而 EvoEnv 仍然带来提升。这表明, 一旦静态验证分布变得饱和或错配, 仅有稳定奖励并不足够; 训练环境也必须随求解器共同演化。第三, 增益并不限于训练中使用的精确环境。EvoEnv 只在来自十个种子的自生成可执行环境上训练, 而没有使用评估基准中的问题-答案数据。尽管如此, 所得策略仍能在格式不同的外部基准上提升表现, 包括 Qwen3-4B-Thinking 上的 GPQA Diamond (65.2 $\rightarrow$ 70.2) 和 LiveCodeBench (59.0 $\rightarrow$ 63.2)。这表明, 演化的环境池诱导的是可迁移的推理行为, 而不只是记忆合成环境模板。

4.3 训练动态：更低的训练分数可能意味着更健康的前沿

图 3 对比了 EvoEnv 与一个固定环境 RLVR 基线在 Qwen3-4B-Instruct-2507 上进行 100 个 on-policy 步时的表现。这里的显著模式是训练分数与留出准确率之间的反向关系。在 EvoEnv 中, 平均训练分数下降, 是因为生成器持续提高求解器环境池的难度。下降的分数并不表示退化, 而是保留前沿信号的机制: 求解器被反复置于有时能成功、有时会失败的状态区间。固定基线在一个饱和

表 1 三个基座模型和八个基准上的主要结果。“平均”是该行已报告单元格的均值。每个模型块内的最佳结果用 **粗体** 标出。

基座模型	方法	AIME'24	AIME'25	AMC'23	HMMT	B-AIME	Brumo	GPQA	LCB	平均
Qwen3-4B Instruct-2507	未训练	58.8	44.0	87.5	30.0	32.0	56.5	52.5	32.6	49.2
	R-Zero	55.0	43.3	87.5	26.5	27.0	55.2	54.0	33.5	47.8
	DAPO	50.0	44.8	92.5	19.8	22.2	47.1	56.1	33.5	45.8
	RLVE	57.1	44.3	92.5	27.7	32.8	57.9	57.1	32.6	50.3
	EvoEnv	61.9	52.5	95.0	30.0	33.6	61.2	55.0	35.5	53.1
Qwen3-4B Thinking-2507	未训练	86.7	78.8	97.5	56.9	53.1	81.9	65.2	59.0	72.4
	R-Zero	84.2	80.6	97.5	53.5	52.9	79.6	67.2	59.0	71.8
	DAPO	74.0	65.4	97.5	48.8	45.8	68.3	64.6	54.0	64.8
	RLVE	81.9	75.0	95.0	51.5	49.6	75.8	67.7	57.3	69.2
	EvoEnv	86.2	83.0	100.0	60.0	53.6	81.9	70.2	63.2	74.8
Nemotron Cascade-8B	未训练	83.3	71.3	92.5	61.5	55.8	73.8	65.2	64.3	71.0
	R-Zero	83.3	71.5	97.5	55.8	50.1	73.3	58.1	62.1	69.0
	DAPO	78.3	63.5	100.0	40.0	40.6	65.8	61.6	60.6	63.8
	RLVE	79.8	69.4	100.0	55.4	54.3	74.2	66.2	56.6	69.5
	EvoEnv	84.8	72.3	100.0	64.2	57.2	78.8	65.7	62.8	73.2

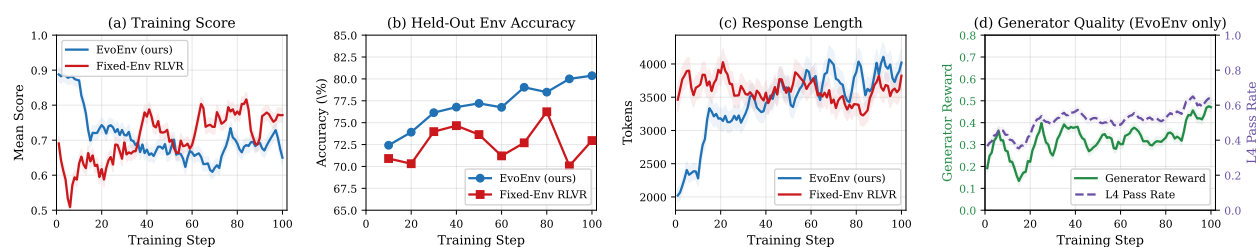


图 3 EvoEnv 与固定环境 RLVR 在 Qwen3-4B-Instruct-2507 上的训练动态。在 EvoEnv 中，随着生成器合成更困难的环境，平均求解器训练分数从 0.88 降至 0.61，而在 50 个未见过的 RLVE 环境上的留出准确率从 72.4% 升至 80.4% [46]。固定基线则停滞在接近 72% 的水平。

池上维持大致恒定的训练分数，并相应地几乎没有留出提升。关键效果在于，环境合成让奖励保持信息量。

4.4 数据层审计：可执行、扩展中且可解释

图 4 给出了三项直接检查。第一，在实际准入流水线下，生成器成为了更好的环境作者：L2 和 L3 通过率从 53.4% 增至约 73.6%，L4 验证从 42.4% 增至 63.0%，并且生成器奖励在前十步和最后十步之间从 0.292 升至 0.452。

第二，被接受的数据流在初始种子扩展之后并未坍塌。从仅有十个种子环境出发，EvoEnv 在 100 步内累计得到 840 个被接受环境，而标签原型数量达到 45。这里，一个 标签原型会在新环境的二元标签向量与每个已有原型的 Jaccard 相似度都低于 $\tau=0.5$ 时被登记，因此该数量给出了结构上不同的环境家族数量的下界。从 10 个种子达到 45 个原型说明，EvoEnv 生成的环境类型远超初始种子分布，而不是回收一组狭窄模板。

第三，仅生成环境的标签热图说明了这些环境为何对 RLVR 训练有用。主要分布集中在可计数、可自动检查的任务上——数论出现在 68.2% 的生成环境中，模结构或序列结构也各约占 38%——但该池还包括可行性、最小化、子集、图/路径、遍历、集合和采样机制。这为求解器提供了许多具有共享可验证性但表面形式各异的可执行推理问题，也从数据侧解释了为什么 EvoEnv 能持续提供未饱和的奖励。

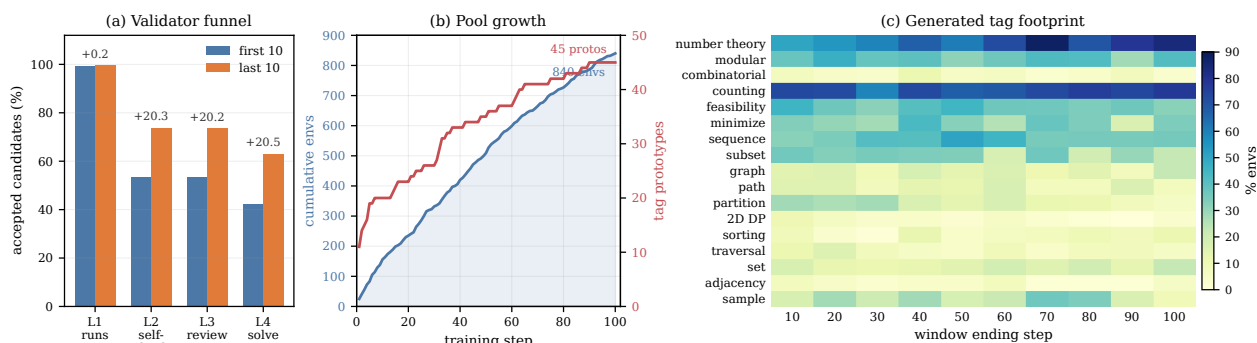


图 4 100 步 Qwen3-4B-Instruct 运行的数据层审计。(a) 生成器被接受的候选在验证漏斗中持续改进：L2/L3 自检和审查通过率提高约 20 个百分点，并且 L4 验证在前十步和最后十步之间从 42.4% 升至 63.0%。(b) 被接受的池持续增长，到第 100 步达到 840 个生成环境和 45 个标签原型。(c) 仅生成环境在 10 步窗口中的语义标签频率；该池包含计数、数论、模结构和序列/子集风格等多种类型的环境，而不是单一复制的种子模板。

表 2 Qwen3-4B-Thinking-2507 上的奖励组件消融。“不含质量”移除验证和难度塑形；“不含多样性”移除由新颖性门控的探索奖励。平均 Δ 是在七个已报告基准上相对于未训练基线的平均提升。

方法	AIME'24	AIME'25	HMMT	B-AIME	Bruno	GPQA	LCB	平均 Δ
未训练	86.7	78.8	56.9	53.1	81.9	65.2	59.0	—
EvoEnv (完整)	86.2	83.0	60.0	53.6	81.9	70.2	63.2	+2.4
不含质量	85.2	80.8	55.2	52.4	82.7	67.2	61.5	+0.5
不含多样性	85.2	81.5	57.3	53.4	81.2	66.7	60.4	+0.6

4.5 奖励组件消融：难度与多样性共同必要

我们隔离两个生成器奖励组件的贡献：质量奖励（验证和难度塑形）与多样性奖励（由新颖性门控的探索奖励）。每个消融都移除其中一个组件，同时保持训练步数、种子和评估协议与 Qwen3-4B-Thinking-2507 上完整 EvoEnv 运行完全相同。两个组件都很重要。移除质量奖励会使平均增益从 +2.4 降至 +0.5，其中 HMMT、GPQA 和 AIME 2025 上的下降最大。移除多样性奖励会把增益降至 +0.6，并且尤其损害 GPQA 和 LiveCodeBench，这表明模板广度是分布外迁移的驱动因素，而不只是表面的正则化器。二者的交互是直观的：没有多样性的难度会产生狭窄能力，而没有难度的多样性会产生许多无法提供梯度的环境。

5 结论

EvoEnv 指出了一条通往如下模型的路径：模型在保留稳定奖励的同时，构建其自身训练环境的一部分。关键并不是信任模型的答案，而是向模型索取可执行结构；这种结构可以被验证、冻结、采样并复用。在零数据推理 RL 中，这把自我改进从一个问题生成循环转变为一个环境构造循环。一次编译、多次求解是其机制；更宽泛的主张是，稳定的自我改进可能取决于模型学会构造训练它们的世界。

参考文献

- [1] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. 2026. URL <https://arxiv.org/abs/2505.23281>.
- [2] Hyunjoo Chae, Jungsoo Park, and Alan Ritter. Safe and scalable web agent learning via recreated websites, 2026. URL <https://arxiv.org/abs/2603.10505>.
- [3] Jiaqi Chen, Bang Zhang, Ruotian Ma, Peisong Wang, Xiaodan Liang, Zhaopeng Tu, Xiaolong Li, and Kwan-Yee K Wong. Spc: Evolving self-play critic via adversarial games for llm reasoning, 2025.
- [4] Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Self-questioning language models, 2025.
- [5] Yixing Chen, Yiding Wang, Siqi Zhu, Haoifei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. Multi-agent evolve: Llm self-improve through co-evolution, 2025.
- [6] Zhaorun Chen, Zhuokai Zhao, Kai Zhang, Bo Liu, Qi Qi, Yifan Wu, Tarun Kalluri, Sara Cao, Yuanhao Xiong, Haibo Tong, Huaxiu Yao, Hengduo Li, Jiacheng Zhu, Xian Li, Dawn Song, Bo Li, Jason Weston, and Dat Huynh. Scaling agent learning via experience synthesis, 2025. URL <https://arxiv.org/abs/2511.03773>.
- [7] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models, 2024.
- [8] Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. Webevolver: Enhancing web agent self-improvement with coevolving world model, 2025. URL <https://arxiv.org/abs/2504.21024>.
- [9] Wenkai Fang, Shunyu Liu, Yang Zhou, Kongcheng Zhang, Tongya Zheng, Kaixuan Chen, Mingli Song, and Dacheng Tao. Serl: Self-play reinforcement learning for large language models with limited data, 2025.
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [11] Bingxiang He, Yuxin Zuo, Zeyuan Liu, Shangzhi Zhao, Zixuan Fu, Junlin Yang, Cheng Qian, Kaiyan Zhang, Yuchen Fan, Ganqu Cui, et al. How far can unsupervised rlvr scale llm training?, 2026.
- [12] Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordani, and Rishabh Agarwal. V-star: Training verifiers for self-taught reasoners, 2024.
- [13] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data, 2025. URL <https://arxiv.org/abs/2508.05004>.

- [14] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. 2024. URL <https://arxiv.org/abs/2403.07974>.
- [15] Jakub Grudzien Kuba, Mengting Gu, Qi Ma, Yuandong Tian, Vijai Mohan, and Jason Chen. Language self-play for data-free training, 2025. URL <https://arxiv.org/abs/2509.07414>.
- [16] Wai-Chung Kwan, Joshua Ong Jun Leang, Pavlos Vougiouklis, Jeff Z Pan, Marco Valentino, and Pasquale Minervini. Opensir: Open-ended self-improving reasoner, 2025.
- [17] Zixing Lei, Sheng Yin, Yichen Xiong, Yuanzhuo Ding, Wenhao Huang, Yuxi Wei, Qingyao Xu, Yiming Li, Weixin Li, Yunhong Wang, and Siheng Chen. Embomatrix: A scalable training-ground for embodied decision-making, 2025. URL <https://arxiv.org/abs/2510.12072>.
- [18] Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, et al. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning, 2025.
- [19] Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning, 2025.
- [20] Mickel Liu, Liwei Jiang, Yancheng Liang, Simon Shaolei Du, Yejin Choi, Tim Althoff, and Natasha Jaques. Chasing moving targets with online self-play reinforcement learning for safer language models, 2025.
- [21] Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models, 2025.
- [22] Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Jiaqi Guo, Haotian Xu, Chutian Wang, Haonan Chen, Xiaoxi Jiang, and Guanjin Jiang. Search self-play: Pushing the frontier of agent capability without supervision, 2025.
- [23] Ivan Moshkov, Darragh Hanley, Ivan Sorokin, Shubham Toshniwal, Christof Henkel, Benedikt Schifferer, Wei Du, and Igor Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. 2025. URL <https://arxiv.org/abs/2504.16891>.
- [24] Archiki Prasad, Weizhe Yuan, Richard Yuanzhe Pang, Jing Xu, Maryam Fazel-Zarandi, Mohit Bansal, Sainbayar Sukhbaatar, Jason Weston, and Jane Yu. Self-consistency preference optimization, 2024.
- [25] Ram Ramrakhya, Andrew Szot, Omar Attia, Yuhao Yang, Anh Nguyen, Bogdan Mazouze, Zhe Gan, Harsh Agrawal, and Alexander Toshev. Scaling synthetic task generation for agents via exploration, 2025. URL <https://arxiv.org/abs/2509.25047>.
- [26] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. 2019. URL <https://arxiv.org/abs/1908.10084>.

- [27] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. 2023. URL <https://arxiv.org/abs/2311.12022>.
- [28] ByteDance Seed, Jiaze Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wenyuan Xu, et al. Seed1. 5-thinking: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*, 2025.
- [29] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, et al. Spurious rewards: Rethinking training signals in rlvr, 2025.
- [30] Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models, 2023.
- [31] Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.
- [32] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [33] Boxin Wang, Chankyu Lee, Nayeon Lee, Sheng-Chieh Lin, Wenliang Dai, Yang Chen, Yangyi Chen, Zhuolin Yang, Zihan Liu, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nemotron-cascade: Scaling cascaded reinforcement learning for general-purpose reasoning models. 2026. URL <https://arxiv.org/abs/2512.13607>.
- [34] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions, 2019. URL <https://arxiv.org/abs/1901.01753>.
- [35] Shaobo Wang, Zhengbo Jiao, Zifan Zhang, Yilang Peng, Xu Ze, Boyu Yang, Wei Wang, Hu Wei, and Linfeng Zhang. Socratic-zero: Bootstrapping reasoning via data-free agent co-evolution, 2025.
- [36] Yiming Wang, Da Yin, Yuedong Cui, Ruichen Zheng, Zhiqian Li, Zongyu Lin, Di Wu, Xueqing Wu, Chenchen Ye, Yu Zhou, and Kai-Wei Chang. Llms as scalable, general-purpose simulators for evolving digital agent training, 2025. URL <https://arxiv.org/abs/2510.14969>.
- [37] Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play swe-rl, 2025. URL <https://arxiv.org/abs/2512.18552>.
- [38] Haoze Wu, Cheng Wang, Wenshuo Zhao, and Junxian He. Mirage or method? how model-task alignment induces divergent rl conclusions, 2025. URL <https://arxiv.org/abs/2508.21188>.
- [39] Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchu Tseng, Zhaoyang Yu, Liang Chen, Yuyao Zhai, Bang Liu, Chenglin Wu,

- and Yuyu Luo. Autowebworld: Synthesizing infinite verifiable web environments via finite state machines, 2026. URL <https://arxiv.org/abs/2602.14296>.
- [40] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning, 2025.
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report, 2025.
- [42] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale, 2025.
- [43] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2024.
- [44] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning, 2022.
- [45] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- [46] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, Chenyang Zhao, Yulia Tsvetkov, Simon Shaolei Du, Natasha Jaques, Hao Peng, Pang Wei Koh, and Hannaneh Hajishirzi. RLve: Scaling up reinforcement learning for language models with adaptive verifiable environments, 2025. URL <https://arxiv.org/abs/2511.07317>.
- [47] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2026. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- [48] Qingyang Zhang, Haitao Wu, Changqing Zhang, Peilin Zhao, and Yatao Bian. Right question is already half the answer: Fully unsupervised llm reasoning incentivization, 2025.
- [49] Zhengxin Zhang, Chengyu Huang, Aochong Oliver Li, and Claire Cardie. Better llm reasoning via dual-play, 2025.
- [50] Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. Infiniteweb: Scalable web environment synthesis for gui agent training, 2026. URL <https://arxiv.org/abs/2601.04126>.
- [51] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shen-zhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data, 2025. URL <https://arxiv.org/abs/2505.03335>.
- [52] Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. Learning to reason without external rewards, 2025.

- [53] Yifei Zhou, Sergey Levine, Jason E Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents, 2026. URL <https://openreview.net/forum?id=9yusqX9DpR>.
- [54] Yujun Zhou, Zhenwen Liang, Haolin Liu, Wenhao Yu, Kishan Panaganti, Linfeng Song, Dian Yu, Xiangliang Zhang, Haitao Mi, and Dong Yu. Evolving language models without labels: Majority drives selection, novelty promotes variation, 2025.
- [55] Yiqi Zhu, Apurva Gandhi, and Graham Neubig. Training versatile coding agents in synthetic environments, 2025. URL <https://arxiv.org/abs/2512.12216>.
- [56] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, et al. Ttrl: Test-time reinforcement learning, 2025.

A 相对于相近自我改进方法的详细定位

表 3 将第 2 节中的概略说明展开为逐类比较。两个轴将 EvoEnv 与每个相邻方法族区分开来：自我改进所合成的单元（一个问题、一个验证器、一条轨迹，或一个可复用环境），以及奖励来源相对于同一策略后续更新的稳定性。策略耦合、学习得到或按实例给出的奖励来源仍可驱动有用的课程，但它本身并不提供可校准、可复用且能多次采样的持久对象。

表 3 EvoEnv 相对于相近自我改进方法的详细定位。“合成单元”是方法在多次更新之间产生并复用的内容；“奖励稳定性”概括被接受的奖励是否依赖当前策略的信念；“代表性方法”仅作示例，并非穷尽。

方法族	合成单元	奖励 / 预言机来源	奖励稳定性	代表性方法	与 EvoEnv 的对比
自提问 / 自指令	一个题目、问题或指令	多数投票、自奖励或学习得到的评判器	策略耦合	R-Zero, SeRL, SQLM, TTRL, EMPO, Intuitor, OpenSIR [13, 9, 4, 56, 48, 52, 16]	自适应课程, 但正确性会随学习者变化。
双人博弈 / 共同演化	教师、提议者、求解器或评判器角色	生成答案、评判器反馈或共同训练信号	大多策略耦合	PasoDoble, Socratic-Zero, MAE, EVOL-RL [49, 35, 5, 54]	课程压力大; EvoEnv 将被接受的奖励外部化为可执行工件。
按实例的可执行自博弈	程序/输入/输出三元组或按任务验证器	针对生成任务或实例的 Python 执行	每个实例固定, 一次 rollout 后即被消耗	AZR, SCA, SPC, Agent0, SWE-Playground, SSR [51, 53, 3, 40, 55, 37]	有落地依据但限于单实例; EvoEnv 验证并复用环境级预言机。
语料库锚定的自博弈	文档锚定任务	检索文本锚定	每个语料库稳定	SPICE, Search Self-Play [19, 22]	为知识任务提供互补锚定; EvoEnv 对算法任务使用代码锚定。
游戏 / 固定规则自博弈	固定规则中的轨迹	游戏结果或固定模拟器	稳定; 规则已给定	SPIRAL, Self-RedTeam, LSP [18, 20, 15]	规则已给定; EvoEnv 合成规则等价工件。
手工构造环境 RLVR	环境	人工编写的评分器	每个环境固定	RLVE [46]	训练单元相同, 但由人工编写而非自合成。
工具 / 智能体环境合成	工具沙盒和场景	基于规则的轨迹验证	流水线后固定	EnvScaler, AutoWeb-World, InfiniteWeb, VeriEnv, AutoPlay, EmboMatrix [31, 39, 50, 2, 25, 17]	流水线构建的智能体沙盒; EvoEnv 是在策略上的推理-环境共同演化。
世界模型模拟器	学习得到的模拟器状态转移	作为转移 / 世界模型的 LLM	依赖学习得到的模拟器	DreamGym, UI-Simulator, WebEvolver [6, 36, 8]	奖励依赖学习得到的模拟器; EvoEnv 的奖励依赖 Python 执行。
开放式演化	智能体或环境共同种群	种群级适应度	随种群变化	POET, Darwin Gödel Machine [34, 47]	更宽泛的开放式议程; EvoEnv 经过求解器校准、在策略上运行且经过验证。
基础性自我改进 (谱系)	生成的推理依据、自生成偏好或过滤后的轨迹	验证器、学习得到的奖励模型或自偏好	多为策略耦合	STaR, ReST-EM, SPIN, Self-Rewarding LMs, V-STaR [44, 30, 7, 43, 12]	从带标签种子启动; EvoEnv 将这一谱系扩展到环境级对象。
EvoEnv (本文)	可复用的可验证环境	固定的 Python 执行	池更新之间固定	—	—

B 可验证环境的剖析

图 5 描绘了单个环境内部的数据流。参考答案由确定性 Python 执行计算，并通过代码路径抵达评分器；求解器只看到渲染后的自然语言提示。这正是验证流水线（附录 C 中的 L1–L5）必须保护的图景：每个防护都确保这些箭头之一继续表达其表面上所表达的含义。

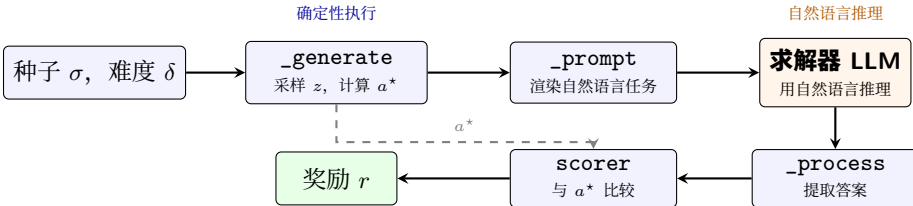


图 5 可验证环境的剖析。参考答案沿代码路径从生成流向评分；求解器只看到渲染后的自然语言任务。

C 附加接口和验证器细节

正文将每个候选描述为 VerifiableEnvironment 的子类。表 4 总结了候选进入池之前使用的验证层。各层有意从廉价的语法检查排列到更昂贵的语义检查和面向求解器的测试，因此早期失败不会消耗求解器 rollout 预算。

表 4 验证器层级。

层级	检查项
L1	可解析的 Python；预期的类和方法存在。
L2	类可实例化；生成、提示渲染、解析和评分能在多个种子与难度值上运行。
L3	相同种子下具确定性；提示、状态和参考答案稳定。
L4	跨种子和难度值的非平凡性；提示和答案会变化。
L5	评分器契约：参考答案得正分；扰动答案、畸形答案和类型不匹配不得正分；解析器不会泄漏隐藏参考。

D 带预植解的子集和环境示例

下面这个最小可行性检查示例展示了环境接口。生成器预植一个解并加入干扰项。求解器必须找到任意有效子集，而评分器通过验证目标和以及多重集重数来检查所声明的约束。

E 针对外部审查器的语义审查审计

第 3.2 节中的同策略审查器只负责把控池准入；它的判定从不进入生成器奖励（式 equation 5）。即便如此，如果该过滤器与更强审查器的一致性不比随机更好，那么它就只是表面装饰，因此我们对其进行离线审计。

设置 我们取用早期训练运行中已经通过全部五个机械层 ($\ell(e) = 5$) 的 79 个环境。真值标签来自 GPT-5.4 的一次审查 (35 个 has_bugs, 44 个 correct)；GPT-5.4 并非完美预言机，但它是比我们的训练中 4B 策略能力强得多的推理器。被审计的审查器与部署配置一致：相同的 Qwen3-4B Instruct-2507 权重，相同的四步结构化提示（数据流、实例轨迹、算法检查、评分器检查），温度 0.6 下的 $K_{\text{rev}} = 3$ 个样本，并用 any-reject 聚合。


```

from collections import Counter

class SubsetSumEnv(VerifiableEnvironment):
    prompt_template = ("Given the multiset {S}, find a nonempty "
                       "submultiset that sums exactly to {target}. "
                       "Output the elements.")

    def _generate(self, seed, difficulty):
        rng = Random(seed)
        n = 8 + difficulty * 4
        k = rng.randint(2, n // 2)
        solution = [rng.randint(1, 50) for _ in range(k)]
        self.target = sum(solution)
        self.S = solution + [rng.randint(1, 50) for _ in range(n - k)]
        rng.shuffle(self.S)
        self.ref_answer = sorted(solution)

    def _prompt_generate(self):
        return self.prompt_template.format(
            S=self.S, target=self.target)

    def _process(self, response):
        return sorted(parse_int_list(response))

    def scorer(self, parsed):
        if not parsed or sum(parsed) != self.target:
            return 0.0
        have, need = Counter(self.S), Counter(parsed)
        if any(need[x] > have[x] for x in need):
            return 0.0
        return 1.0

```

图 6 一个可行性检查环境。存储的参考答案对验证有用，但评分器接受任何满足所声明约束的子集。

一致性 表 5 报告了四种聚合规则下的一致性。部署的 any-reject 配置达到 $F_1 = 87.0\%$ ($P = 85.7\%$, $R = 88.2\%$, 准确率 88.6%)，显著高于总是输出 `correct` 的基线（准确率 55.7%，零召回）以及总是输出 `has_bugs` 的基线（精确率 44%）。

表 5 同策略审查器（Qwen3-4B-Instruct, $K_{\text{rev}} = 3$, $T = 0.6$ ）与 GPT-5.4 在 79 个带可解析判定的环境上的一致性（其中按 GPT-5.4, 35 个有 bug, 44 个正确）。

聚合方式	准确率	精确率	召回率	F_1	(TP, FP, TN, FN)
$K = 1$ (单次审查)	65.8%	75.0%	34.3%	47.1%	(12, 4, 40, 23)
$K = 3$, 多数表决	65.8%	78.6%	31.4%	44.9%	(11, 3, 41, 24)
$K = 3$, 任一拒绝	88.6%	85.7%	88.2%	87.0%	(30, 5, 40, 4)
$K = 3$, 全部拒绝	63.3%	87.5%	20.0%	32.6%	(7, 1, 43, 28)

失败模式 一个均衡的人工案例研究与量化图景相符。审查器能够可靠捕捉用贪心替代最优算法的情况、硬编码或可平凡推导的参考答案、数学性质违规（例如把非素数宣称为素数），以及大多数会在小型追踪实例上暴露的错误递推 bug。它最薄弱的地方是跨方法数据流（在 `_generate` 中写入但从未由 `_prompt_generate` 暴露的参数，在逐方法隔离审查时很难发现）、微妙的数据生成病态（例如随机移除边而悄然破坏连通性假设），以及对原本正确代码的幻觉式边界情况。这三类模式解释了表 5 中大多数 FN 和 FP。

为什么这个门控已足够 残余分歧不会直接污染生成器梯度：判定只进入式 equation 7，不进入式 equation 5。被误报为有问题的正确候选仍会收到其机械的 $Q_{\text{val}} + \gamma_t N_t$ 奖励，只是失去求解器

训练池中的位置；而被漏报的有 bug 候选在能够训练求解器之前，仍会被求解器相对通过率条件 $0 < \hat{a}_m < 1$ 过滤。因此，审查器只需要相对于外部参考具有正确偏向，而不需要与其完全一致。

F 种子环境

所有运行都从同一组十个初始种子环境开始。这些种子是可执行接口示例和初始池条目，而不是外部问题—答案数据集：它们定义文件布局、参数约定、提示渲染约定和评分器约定，供生成器模仿，同时让生成器在训练期间合成新的环境代码。我们有意选择这十个种子，以覆盖 (i) 不同算法类别，(ii) 不同预言机结构（精确匹配、逐元素匹配、部分得分比例和可行性），以及 (iii) 不同难度旋钮（问题规模、取值范围、结构密度、递归深度）。这套精确的十种子集合在所有实验中保持固定。

表 6 逐一列出了每个种子。每一行对应一个实现附录 B 中 VerifiableEnvironment 接口的 Python 类；参数旋钮是生成器通过 self.parameter 填入的键，而评分器列则给出该种子所代表的评分器族。我们在表后进一步展开这四个评分器族。

表 6 十个初始种子环境。每一行给出环境类名、它代表的算法类别、要求求解器完成的任务、暴露给生成器的难度旋钮（从 self.parameter 读取），以及该种子所代表的评分器族（在表下方定义）。

#	种子类	类别	任务	难度旋钮	评分器族
1	Sorting	数组 / 排序	将 N 个整数按升序排序。	N ，取值范围	逐元素
2	SlidingWindow	数组 / 双端队列	输出每个连续大小为 K 的窗口的最小值。	N, K	逐元素
3	MonotonicStack	数组 / 栈	对每个 i ，统计满足 $A[i] > \max(A[i+1..j])$ 的 $j > i$ ，然后求和。	N	精确匹配
4	Knapsack	0/1 动态规划 / 优化	在重量预算下选择物品，使总价值最大。	$N, W_{\max}$ ，取值范围	部分得分比例
5	SubsetSum	组合 / 可行性	选择一组索引，使其和恰好等于预值目标。	N	可行性
6	BoundedIntervalIntersection	组合 / 计数	统计交集长度 $\geq K$ 的非空区间子集数。	N, K	部分得分比例
7	Bridge	图 / 连通性	找出无向图中的所有桥边。	N ，连通分量数，边密度	部分得分比例
8	EuclidGame	博弈论 / 数论	判定 (X, Y) 上欧几里得减法游戏在最优博弈下的胜者。	$\max(X, Y)$	精确匹配（二元）
9	Fibonacci	线性递推	计算 $A[N] \bmod m$ ，其中 $A[n] = P \cdot A[n-1] + Q \cdot A[n-2]$ 。	N ，模数	精确匹配
10	RecursiveFunction	递归 / Ackermann 类	按含自嵌套调用的三分支递推计算 $f(M, N)$ 。	$\max(M, N)$	精确匹配

为什么十个种子已经足够 尽管数量很小，这十个种子仍让生成器接触到下游使用的每一种接口模式。它们横跨七个算法族、四个评分器族，以及覆盖数组、图、递推、最优子结构（DP）和博弈论状态的结构原语。四个种子（EuclidGame, Fibonacci, MonotonicStack, SlidingWindow）改编自程序设计竞赛归档（Luogu OI）；其余六个从零编写，用于平衡评分器族和结构覆盖。第 4.4 节和附录 G.1 报告了训练期间生成器从这一起始集合出发所遵循的轨迹。

G 生成环境数据审计

G.1 100 步运行中的定性示例

下面的示例在环境层面说明了同样的转变。我们将这个示例集限制为不晚于第 100 步生成的环境，使其与第 4.4 节中的消融研究相匹配。距离是环境标签集到十个原始种子中最近者标签集的 Jaccard 距离。

表 7 100 步运行中的代表性生成环境。为便于阅读，提示被缩短为其第一个分句。

类	步数	d_{seed}	# 标签	提示草稿
PrimePartition	1	0.73	8	给定一个正整数列表，计数或判定受素数约束的划分。
PrimePairProductSum	10	0.71	6	给定 N 和一组互异素数，优化一个乘积/求和表达式。
PrimeGCDChain	60	0.61	17	将一串值与 GCD 和素数链约束结合。
SubsetCycleGCD_PrimeChain	69	0.65	16	融合子集选择、循环结构以及 GCD/素数约束。
CycleDistanceSum	83	0.75	9	对类环组合结构诱导的距离进行推理。

这些示例并非意在通过人工检查来证明正确性；每个被接受的环境仍然要通过第 3.2 节描述的验证和自审查流水线才能准入。它们的目的在于使分布审计具体化：生成器会快速从种子池转向数论、模运算和序列结构问题，这些问题经过求解器校准，而不是从 RLVE 中平凡复制。

H 局限性与风险

环境接口的适用范围 本文研究的是一个受控设定：使用确定性 Python 环境进行推理的零数据 RLVR。当前接口最适合那些可紧凑编写并自动验证可执行采样器、预言机、渲染器和评分器的任务。这包括许多算法式和可行性风格任务，但并不能为开放式判断、人类偏好建模、长时程物理模拟或交互式工具使用环境建立同样的结论。将可验证环境合成扩展到这些设定，需要不同的锚定机制和更强的安全检查。

可执行代码安全 要求模型合成可执行环境的方法，必须将生成代码视为不可信。我们的实现会在带有受限导入、资源限制、确定性种子和挂钟超时的沙箱化子进程中执行候选环境；违反接口或执行约束的候选环境会在进入池之前被拒绝。这些防护是方法本身的一部分，而不是可选的工程细节。任何向更丰富的工具使用、Web、文件系统或具身环境的扩展，都需要相应更强的隔离和监控。

更广泛影响 可验证环境合成的一个积极含义是，训练课程变得更可检查：被接受的环境是具体的代码工件，而不是不透明的伪标签或隐藏的偏好判断。这可以减少对专有后训练数据集的依赖，并使奖励来源更容易审计。同时，改进自主课程构建也可能在有益场景和双重用途场景中同时放大推理模型的能力。因此，我们认为，透明地发布环境、沙箱化执行以及仔细审计生成工件，是后续工作的必要防护。

I 超参数

表 8 列出了正文中报告的主要超参数；表 9 按组件分组报告了已部署训练运行的完整配置。

表 8 正文中报告的训练与评测超参数。

超参数	取值
生成器优势缩放 w_{gen}	0.3
用于候选校准的求解器测试预算	$m = 8$
语义自审样本数	$K_{\text{rev}} = 3$ ，采用任一拒绝聚合
提示/代码新颖性混合 λ	0.5
原始种子下限 ρ_{min}	0.2
训练步数	100 步
评测上下文	最高 128k
评测温度 / top- p	0.6 / 0.95

表 9 已部署 4B-Instruct 训练运行的完整配置。生成器组、求解器组、新颖性、池、自审和沙箱设置在各骨干变体之间共享；扩展到 8B 时，仅重缩放优化器块中的批大小和上下文长度。

块	超参数	取值
生成器组	每步不同生成器提示的数量	16
	GRPO 组大小 M （每个生成器提示的响应数）	8
	生成器采样温度	1.0
	生成器最大响应长度	8k 个词元（rollout 侧）
求解器组	求解器批大小 B （ <code>data.train_batch_size</code> ）	64
	PPO 小批大小	64
	GRPO 组大小（ <code>rollout.n</code> ，每个提示的响应数）	8
	求解器采样温度	1.0
	求解器最大提示 / 响应长度	8k / 16k 个词元
新颖性调度	新颖性权重边界（ $\gamma_{\text{min}}, \gamma_{\text{max}}$ ）	(2.0, 5.0)
	激活阈值（ $\tau_{\text{low}}, \tau_{\text{high}}$ ）	(0.45, 0.65)
	入池门控 τ_{gate}	0.80
	$\bar{s}_t$ 的 EMA 衰减	$\alpha_{\text{ema}} = 0.6$ （初始 0.5）
	嵌入模型	all-MiniLM-L6-v2（冻结）
	提示/代码混合 λ	0.5
校准 & 接纳	每个 L5 候选的校准实例数 m	最高 8（式 equation 1）
	每个校准实例的求解器响应数	1
池轮换	每个环境的池最大轮次数	5
	池轮换节奏	每 10 个训练步
	原始种子下限 ρ_{min}	0.2
	初始池	十个种子（附录 F）
自审	独立审查数 K_{rev}	3
	聚合规则	任一拒绝
	审查温度	0.6
	审查最大响应长度	8k 个词元
RL 目标 & 优化器	算法	GRPO
	PPO 裁剪比率（低 / 高）	0.2 / 0.28
	双 clip 常数 c	10.0
	KL 损失系数 β	1×10^{-3}
	熵系数	0
	优化器	AdamW
	学习率 / 预热 / 权重衰减	5×10^{-7} / 5 步 / 0.1
	梯度裁剪	1.0
沙箱 & 代码	验证子进程挂钟超时	每层 30 s
	允许导入（通过生成器提示强制执行）	<code>random, math, collections, itertools, heapq, bisect, functools, re, typing</code>
	禁止导入 代码抽取	任何第三方库（例如 <code>numpy, networkx, scipy</code> ） 最长的 Python 代码围栏块